Few-Step Distillation of SD3.5-M

How we got here, and the Phase R result

Hesam Asadollahzadeh

30 July 2026

The setting

Goal. Turn a 28-step SD3.5-Medium teacher into a **4-step student** — a 7$\times$ speed-up — without giving up image quality or prompt alignment.

Two numbers we live by

- **CMMD** vs. real COCO images — fidelity. *Lower is better.*
- **VQAScore** — prompt alignment. *Higher is better.*

Our reference student: "M1" — pure consistency distillation, 20k steps.

at 4 steps	CMMD	VQAScore
base SD3.5-M	161.3	—
M1 (ours)	**69.7**	**0.7848**
teacher, 28 steps	54.1	0.80

CMMD on 640 images is an *internal gate*, not publication FID (needs 5–10k).

Previous result 1 — the teacher must be frozen

A 15-run campaign on why our self-distillation kept producing garbage.

Root cause. The student was trained only to match **teacher = EMA(student)**. That objective has **no fixed point** — nothing pins the output scale, so student and target inflate together.

teacher	result	
EMA, decay 0.999	diverges ∼step 2k	**garbage**
EMA, decay 0.9999	diverges ∼step 20k	**delay, not cure**
frozen	flat for the whole run	**works**

And a retraction. A KL-to-base anchor *looked* like it rescued the EMA design — scale held flat for 20k steps. We finally scored its images: **CMMD 467** vs. M1's 69.7.

Lesson

Stable training diagnostics are **necessary but not sufficient**. Never accept a fix without scoring generated images.

Previous result 2 — our scoring machinery did nothing

The original method scored trajectories with a learned projector into CLIP space, plus a reward term and an alignment anchor.

We ran it against pure consistency distillation, everything else identical:

	M1 (pure consistency)	M2 (full method)
consistency loss *d*	59–61	59–61
output scale	∼350	∼350
CMMD @ 4 steps	69.7	69.1

Identical at every step, within 1–3%. The projector, the reward, the CLIP towers and a 118k-image feature precompute **changed nothing**.

This is why the current runs use `--latent_matching_baseline`: plain consistency distillation, and we add *one* thing at a time.

Previous result 3 — the four-channel decomposition

Separate question, same codebase: picking the best of N samples with a verifier clearly helps at inference. **Can a student absorb that gain?**

channel	effect on the primary metric	
Teacher best-of-4 at inference	+0.0717	the prize
Input (which prompts/states)	+0.0004	**null**
Target (which endpoint to copy)	−0.0222	**harmful**
Noise (which seed)	+0.0485 available	**real headroom**
...but learnable?	3 attempts, all null	**not amortisable**

Four candidate mechanisms for *why* the target channel backfires were proposed and then **refuted by our own controls**. The decomposition stands; the explanation does not.

So we needed a second, independent line of attack on student quality.

What redirected us: a motivation that failed a measurement

The story we were going to tell. "High guidance buys prompt alignment but ruins image quality — so fix it with representation alignment."

We measured the curve first. At 28 steps:

	fidelity optimum	alignment optimum
guidance scale	CFG 3	CFG 7
CMMD	54.1	55.5

A **1.4 CMMD** gap. **There is no trade-off worth a method there.** Motivation dropped.

But the same measurement exposed a real gap:

our 4-step student **69.7** vs. teacher's best **54.1** $\Rightarrow$ **15.6 CMMD to close**

That gap — not the guidance trade-off — is the honest target.

Phase R: what we are doing now, and why

REPA (REPresentation Alignment) — its published strength is exactly **FID**, which is the axis of our 15.6 CMMD gap.

The idea. While the denoiser processes a *noised real image*, take an intermediate hidden state (block 8 of 24), project it, and align it to a frozen **DINOv2** encoder's features of the *clean* image:

$$L_{\text{repa}} \;=\; 1 - \cos\big(\, P(h_8(z_t)),\; \text{DINOv2}(x_{\text{clean}})\,\big)$$

- Runs on **real data**, in its own forward pass — independent of the distillation trajectory.
- Weight $\lambda_{\text{repa}} = 0.5$ (REPA's own default).
- Everything else is **M1's exact recipe**, so **M1 is the control**.

Pre-registered read — fixed before the run

CMMD < 69.7 *and* VQA ≥ 0.7848	real win: alignment closes part of the gap
CMMD ≈ 69.7	negative: REPA does not transfer to few-step
VQA < 0.7848	we moved *along* the trade-off, not past it

The first attempt broke — and not for a scientific reason

Job 91058 ran 15,000 steps over 10 hours and produced **flat saturated colour blocks**.

at 4 steps	CMMD	VQAScore
M1 (control)	69.7	0.7848
job 91058	**502.1**	**0.2519**

Its script claimed "*EXACT M1 control — the only difference is `--lambda_repa`*". Diffing the two saved configs, **four live hyperparameters** had also moved, because the script passed none of them and inherited **parser defaults that had drifted** since M1 was run:

flag	M1	91058	controls
`--freeze_teacher`	True	**False**	frozen vs. live EMA teacher
`--guidance_scale`	7.0	**0.0**	teacher target quality
`--scoring_window`	0.4,0.9	**0.3,0.7**	which noise levels are trained
`--delta_max`	3	**2**	student's timestep gap

The header even listed "ema 0.999" as copied — true, but *inert* for M1, because `--freeze_teacher` skips the EMA entirely. The inert knob was copied; the live one was dropped.

Why nobody noticed for ten hours

1. The loss could not see the damage. With teacher = EMA(student), the consistency loss compares the student to *its own running average*. Damage that moves both is invisible.

	consistency loss d	model
M1 (frozen teacher)	40–92	**good**
job 91058 (EMA)	**20–42**	**destroyed**

The broken run's loss looked *better* throughout.

2. Nobody was looking at the pictures. Image sampling to W&B was switched off, so not one student sample was ever logged.

Also checked and ruled out: this was *not* the old output-scale runaway. Measured 209 $\rightarrow$ 390, while M1 itself swings 258–396. **The ranges overlap.** A model can be completely destroyed with textbook-clean scale diagnostics.

What is running now — job 91342

Same experiment, made honest. **Pure config fix**: all four flags restored, and the flag set is defined *once* and shared, so the test run and the real run cannot drift apart.

Two gates that fail in 10 minutes instead of 11 hours

- **Gate 0** — 300-step smoke: is L_{repa} finite and descending? **PASS** ($0.939 \rightarrow 0.660$)
- **Gate 1** — diff the saved config against M1's: is the *only* difference the treatment? **PASS** (diff = **NONE**)

Plus: student images logged to W&B every 500 steps, at 4 and 28 steps.

The gates already earned their cost

Gate 0 caught a *different* bug on the first try — an "improvement" to the gradient-sync path that crashed at step 1. Cost: 3 minutes of GPU instead of 11 hours.

RESULT — the pre-registered read passes

20 000 steps, 12 h 18 m, completed clean. Evaluated on the identical 640-prompt pool, seeds and scorers as M1.

steps	VQA REPAv2	VQA M1	ΔVQA (95 % CI)	CMMD REPAv2	CMMD M1
4	**0.7910**	0.7848	+0.0063 [−0.0053, +0.0179]	**67.4**	69.7
8	0.8161	0.8114	+0.0047 [−0.0065, +0.0161]	59.1	60.8
28	0.8104	0.8012	+0.0092 [−0.0019, +0.0204]	59.9	60.3

At **4 steps** — the deployment regime — the gap to the teacher's best (54.1) narrows **15.6** $\rightarrow$ **13.3**, closing **14.7 %**, with alignment held.

CMMD moves the right way at all three step counts, and most where it matters most.

The obvious confound — checked and excluded

Our own comparison script warns that a CMMD gain may just mean REPA *damped* the student: a model that moves less stays nearer the base and can look more "realistic" for reasons that have nothing to do with DINOv2.

That warning's numbers are hardcoded from job **91058** — the broken run. Measured for *this* run:

run	final d	final $\Delta_{\theta\xi}$	mean grad
REPAv2 91342	66.0	**42.9**	198.3
M1 control	65.7	**42.9**	195.6
91058 (broken)	32.1	5.5	748.7

Parameter drift **identical to three significant figures**; gradient norm within 1.4 %. The student moved exactly as far as M1's. **Damping does not explain the result.**

Note: REPA changed the learned function while leaving *every* training-dynamics statistic indistinguishable from the control. The loss curves could not have predicted this in either direction.

Reported honestly — what this is not

REPAv2 wins at 4 steps only. On the same pool, M2 (the encoder-scoring method we previously called redundant) is better at 8 and 28 steps:

CMMD vs real COCO	4 step	8 step	28 step
M1 (control)	69.7	60.8	60.3
M2	69.1	**58.2**	**57.2**
REPAv2	**67.4**	59.1	59.9

"Best recipe we have" is **not** supported — best *at 4 steps* is.

Four limits, in order of how much they threaten the claim:

1. **No confidence interval on CMMD.** −2.3 is 3.3 % relative on 640 images and we have no noise floor. A paired bootstrap needs no regeneration — minutes of work.
2. **VQA gains are not significant** — all three CIs cross zero. Alignment *held*; it did not improve.
3. **One seed.** Effects of this size are not separable from seed noise.
4. **Mechanism unknown.** Damping is excluded; nothing else is.

Takeaways

1. **The teacher must be frozen.** Any EMA-of-student teacher eventually collapses; slower decay only delays it.
2. **Score the images, not the loss curves.** Two separate "fixes" looked healthy on diagnostics and produced garbage.
3. **Never trust a parser default as a baseline.** Pass every hyperparameter explicitly and diff the saved config against the reference run — 91058 lost 10 GPU-hours to exactly this.
4. **Cheap gates before expensive runs.** Two 10-minute checks caught two separate bugs that would each have cost a full day.
5. **Result:** REPA on the M1 recipe improves 4-step CMMD **69.7** $\rightarrow$ **67.4** (14.7 % of the gap) with alignment held and the damping confound excluded — but it needs a CMMD confidence interval and a second seed before it is more than suggestive.